21

Geolocation and Maps

In this chapter, you'll learn about the geolocation and mapping capabilities of React Native. You'll start the learning process with how to use the **Geolocation API**, and then you'll move on to using the `MapView` component to plot points of interest and regions. To do this, we'll use the `react-native-maps` package to implement maps.

The goal of this chapter is to go over what's available in React Native for geolocation and in `react-native-maps` for maps.

Here's a list of the topics that we'll cover in this chapter:

- Using the Geolocation API
- Rendering the map
- Annotating points of interest

Technical requirements

You can find the code file for this chapter on GitHub at https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter21.

Using the Geolocation API

The Geolocation API that web applications use to figure out where the user is located can also be used by React Native applications because the same API has been polyfilled. Other than maps, this API is useful for getting precise coordinates from the GPS on mobile devices. You can then use this information to display meaningful location data to the user.

Unfortunately, the data returned by the Geolocation API is of little use on its own. Your code must do the legwork to transform it into something useful. For example, latitude and longitude don't mean anything to the user, but you can use this data to look up something that is of use to the user. This might be as simple as displaying where the user is currently located.

Let's implement an example that uses the **Geolocation API** of React Native to look up coordinates and then use those coordinates to look up human-readable location information from the Google Maps API.

Before we start coding, let's create a project using `npx create-expo-app` and then add the location module:

```
npx expo install expo-location
```

Next, we need to configure location permissions in the app. Accessing a user's location in a mobile app requires explicit permission from the user. Later in this example, we will do that by calling the `Location.requestForegroundPermissionsAsync()` method. This will display a permission dialog to the user asking them to allow or deny location access. It's important to check the status returned to see if permission was granted before proceeding to use location methods. If permission is denied, you should gracefully handle it in your code and potentially prompt the user to grant permission in app settings if needed.

In real apps, before we can request permissions, we should first set those permissions up in the app configuration. We can do this by adding a plugin to the `app.json` file:

```
{
  "expo": {
    "plugins": [
      [
        "expo-location",
        {
          "locationAlwaysAndWhenInUsePermission": "Allow $(PRODUCT_NAME) to use your location."
        }
      ]
```

```
      ]
    }
  }
```

You should request location permission as early as possible, such as when your app first starts up or when the user first navigates to a screen that requires location. By requesting permission up-front and properly handling the user's choice, you can ensure your app works as expected while respecting the user's privacy preferences.

When you have a prepared project, let's have a look at the `App` component, which you can find here: **https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter22/where-am-i/App.tsx**. The goal of this component is to render the properties returned by the Geolocation API on the screen, as well as looking up the user's specific location and displaying it.

To fetch a location from the app, we need to grant permissions. In `App.tsx`, we have called `Location.requestForegroundPermissionsAsync()` for that.

The `setPosition()` function is used as a callback in a couple of places, with its job being to set the state of your component. Firstly, `setPosition()` sets the latitude-longitude coordinates. Normally, you wouldn't display this data directly, but this is an

example that shows the data that's available as part of the Geolocation API. And, secondly, it uses the `latitude` and `longitude` values to look up the name of where the user currently is, using the Google Maps API.

In the example, the `API_KEY` value is empty, and you can get it here: https://developers.google.com/maps/documentation/geocoding/start.

The `setPosition()` callback is used with `getCurrentPosition()`, which is only called once when the component is mounted. You're also using `setPosition()` with `watchPosition()`, which calls the callback any time the user's position changes.

The iOS emulator and Android Studio let you change locations via menu options. You don't have to install your app on a physical device every time you want to test changing locations.

Let's see what this screen looks like once the location data has loaded:


12:28
Address: Camí de la Verneda, 30, Sant Martí, 08020 Barcelona, Spain
Latitude: 41.42500974177705
Longitude: 2.2070894112218014

Figure 21.1: Location data

The address information that was fetched is probably more useful in an application than latitude and longitude data. It works well for apps that need to find buildings around you or companies. Even better than physical address text is visualizing the user's physical location on a map; you'll learn how to do this in the next section.

Rendering the map

The `MapView` component from `react-native-maps` is the main tool you'll use to render maps in your React Native applications. It offers a wide range of tools for rendering maps, markers, polygons, heatmaps, and the like.

You can find more information about `react-native-maps` on the website: **https://github.com/react-native-maps/react-native-maps**.

Let's now implement a basic `MapView` component to see what you get out of the box:

```
import { View, StatusBar } from "react-native";
import MapView from "react-native-maps";
import styles from "./styles";
StatusBar.setBarStyle("dark-content");
export default () => (
  <View style={styles.container}>
    <MapView style={styles.mapView} showsUserLocation followsUserLocation />
  </View>
);
```

The two Boolean properties that you've passed to `MapView` do a lot of work for you. The `showsUserLocation` property will activate the marker on the map, which denotes the physical location of the device running this application. The `followsUserLocation` property tells the map to update the location marker as the device moves around.

Here is the resulting map:


12:31
Nantes
Dijon
FRANCE
Geneva
Lyon
Bordeaux
Grenoble
Toulouse
Bilbao
Marseille
PYRENEES
Zaragoza
SPAIN
Madrid
Valencia
Palma
Murcia
Málaga
Algiers
Oran
TELL ATLAS
MOUNTAINS
Djelfa
Maps

Figure 21.2: Current location

The current location of the device is clearly marked on the map. By default, points of interest are also rendered on the map. These are things close to the user so that they can see what's around them.

It's generally a good idea to use the `followsUserLocation` property whenever using `showsUserLocation`. This makes the map zoom to the region where the user is located.

In the following section, you'll learn how to annotate points of interest on your maps.

Annotating points of interest

Annotations are exactly what they sound like: additional information rendered on top of the basic map geography. You get annotations by default when you render `MapView` components. The `MapView` component can render the user's current location and points of interest around the user. The challenge here is that you probably want to show the points of interest

relevant to your application instead of those rendered by default.

In this section, you'll learn how to render markers for specific locations on the map, as well as rendering regions on the map.

Plotting points

Let's plot some local breweries! Here's how you pass annotations to the `MapView` component:

```
<MapView
  style={styles.mapView}
  showsPointsOfInterest={false}
  showsUserLocation
  followsUserLocation
>
  <Marker
    title="Duff Brewery"
    description="Duff beer for me, Duff beer for you"
    coordinate={{
      latitude: 43.8418728,
      longitude: -79.086082,
    }}
  />
  {...}
</MapView>
```

In this example, we've opted out of this capability by setting the `showsPointsOfInterest` property to `false`. Let's see where these breweries are located:

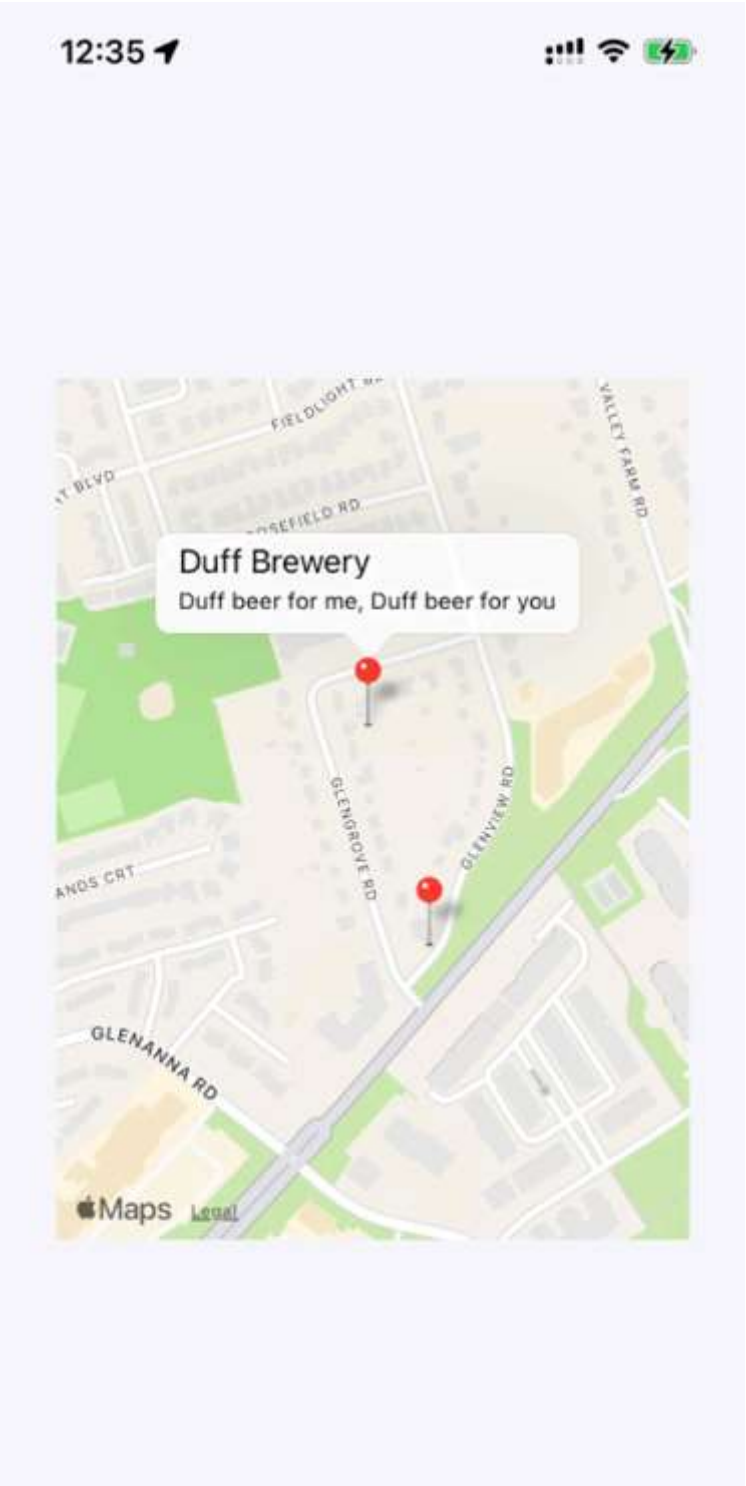
12:35
Duff Brewery
Duff beer for me, Duff beer for you
GLENGROVE RD
GLENVIEW RD
GLENANNA RD
VALLEY FARM RD
Maps
Legal

Figure 21.3: Plotting points

The callout is displayed when you press the marker that shows the location of the brewery on the map. The `title` and `description` property values that you give to `<Marker>` are used to render this text.

Plotting overlays

In this last section of this chapter, you'll learn how to render region overlays. Think of a region as a connect-the-dots drawing of several points, and a point is a single `latitude/longitude` coordinate.

Regions can serve many purposes. In our example, we'll create a region that shows where we're more likely to find IPA drinkers versus stout drinkers. You can follow this link to see what the full code looks like: **https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter22/plotting-overlays/App.tsx**. Here is what the JSX part of the code looks like:

```
      <View style={styles.container}>
        <View>
          <Text style={ipaStyles} onPress={onClickIpa}>
            IPA Fans
          </Text>
          <Text style={stoutStyles} onPress={onClickStout}>
            Stout Fans
          </Text>
        </View>
        <MapView
          style={styles.mapView}
          showsPointsOfInterest={false}
          initialRegion={{
            latitude: 43.8486744,
            longitude: -79.0695283,
            latitudeDelta: 0.002,
            longitudeDelta: 0.04,
          }}
        >
          {overlays.map((v, i) => (
            <Polygon
              key={i}
              coordinates={v.coordinates}
              strokeColor={v.strokeColor}
              strokeWidth={v.strokeWidth}
            />
          ))}
        </MapView>
      </View>
```

The region data consists of several `latitude/longitude` coordinates that define the shape and location of the region. Regions are placed in the `overlays` state variable, which we map into `Polygon` components. The rest of this code is mostly about the handling state when the two text links are pressed.

By default, the IPA region is rendered as follows:

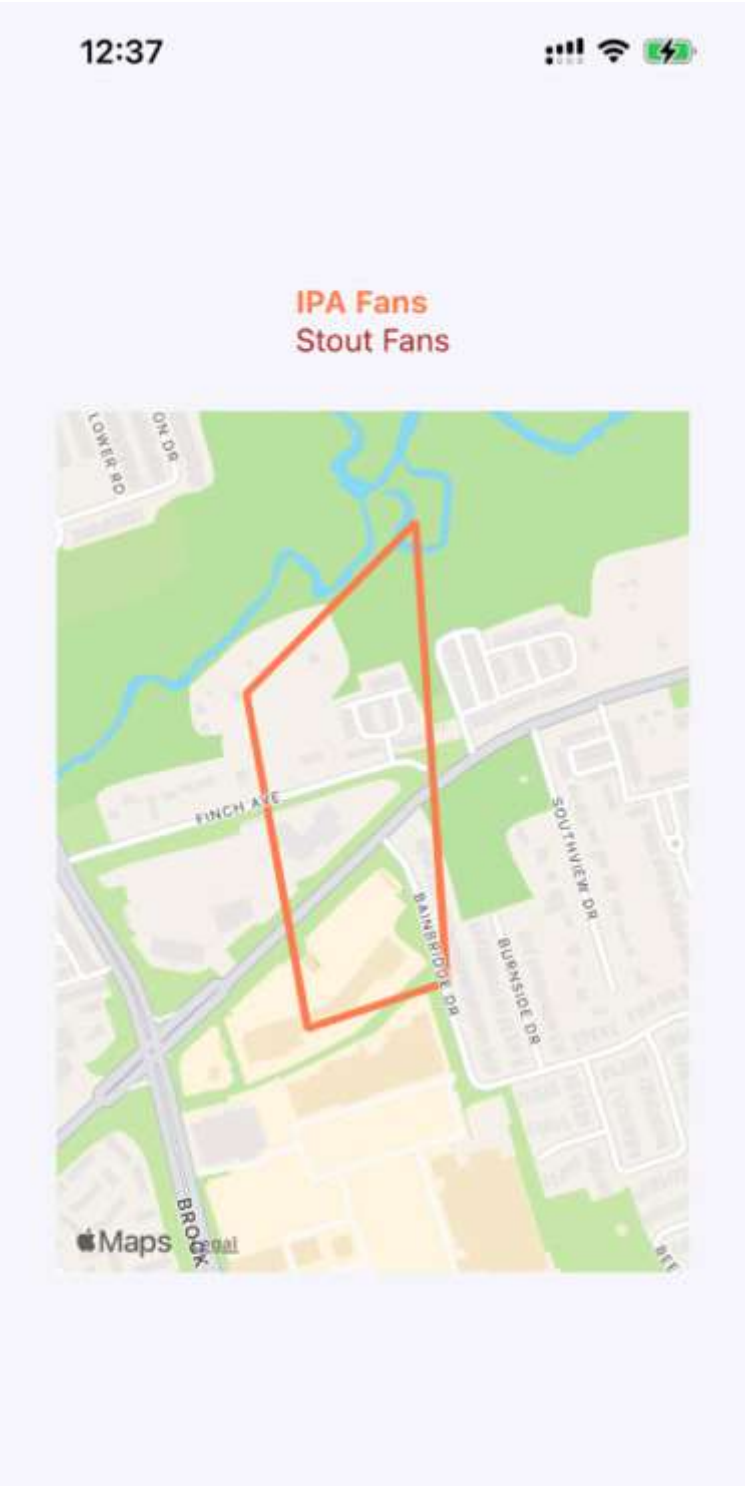
12:37
IPA Fans
Stout Fans
LOWER RD
OAK DR
FINCH AVE
SOUTHVIEW DR
BAINBRIDGE DR
BURNSIDE DR
BROCK
Maps

Figure 21.4: IPA Fans

When the **Stout Fans** button is pressed, the IPA overlay is removed from the map and the stout region is added:

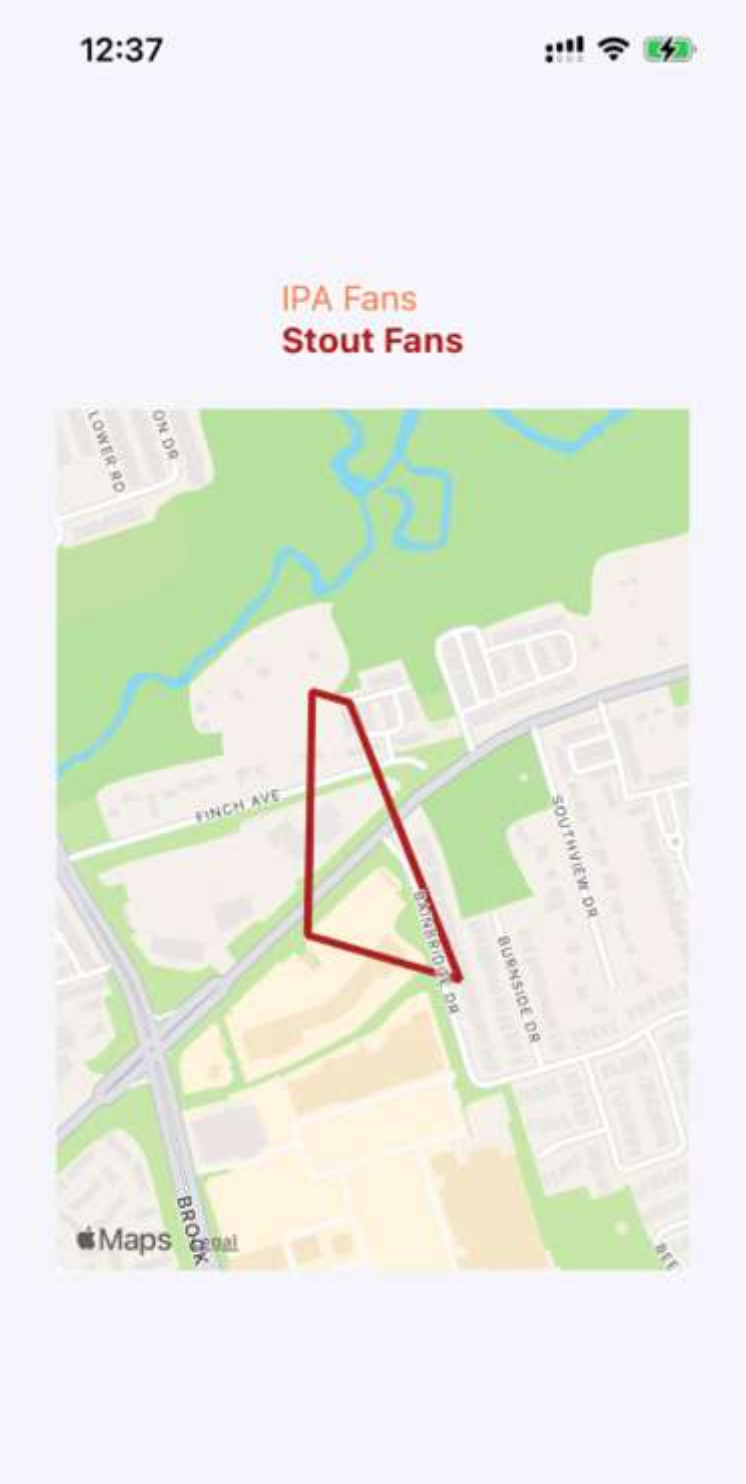
12:37
IPA Fans
Stout Fans
LOWER RD
FINCH AVE
SOUTHVIEW DR
BURNSIDE DR
BROCK
Maps

Figure 21.5: Stout Fans

Overlays are useful when you need to highlight an area instead of a `latitude/longitude` point or an address. As an example, it might be an app for finding apartments for rent in the area or neighborhood you select.

Summary

In this chapter, you learned about geolocation and mapping in React Native. The Geolocation API works the same as its web counterpart. The only reliable way to use maps in React Native applications is to install the third-party `react-native-maps` package.

You saw the basic configuration `MapView` components and how they can track the user's location and show relevant points of interest. Then, you saw how to plot your own points of interest and regions of interest.

In the next chapter, you'll learn how to collect user input using React Native components that resemble HTML form controls.